Audio Visualizer with Hardware SD Audio Streaming

Names: Anthony Montalto, Zeead Sowelam
Course: ECE 385
TA: Feiyang Liu
Date: 12/13/2025

1. Introduction

a. Project Overview

This project implements a standalone music visualizer on the Spartan-7 Urbana board. When initialized, a hardware SD-card loader streams raw 8-bit PCM audio data from an SD card directly into on-chip BRAM. After loading completes, a MicroBlaze plays the samples through a PWM audio peripheral using a hardware-timed interrupt at a fixed sample rate while also updating a real-time visualization over HDMI using the previously developed *Lab 7 text controller*. A USB keyboard is used to change visualization modes and control playback settings, providing the user with a fully fleshed out customizable audio visualizing experience.

b. Relation to Lab 6 and Lab 7 Designs

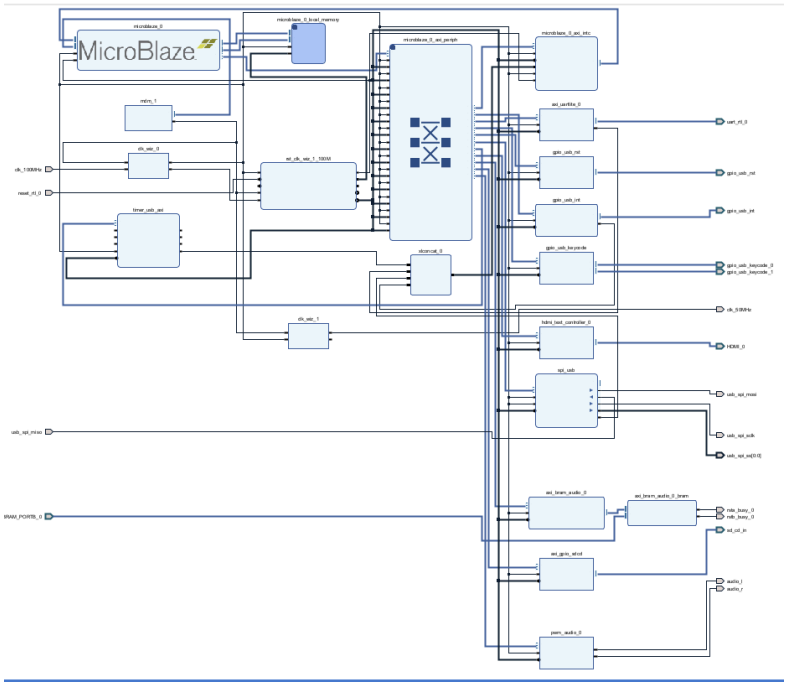
The system reuses the Lab 6 MicroBlaze + USB host architecture for keyboard input and the Lab 7 HDMI text controller infrastructure for display output. The final project extends these designs by adding a dedicated SD-to-BRAM hardware loading path that streams raw audio without filesystem parsing, an interrupt-driven audio playback pipeline that uses an AXI timer ISR to feed the PWM audio output at a fixed sample rate, and a visualization routine that renders from recent samples while keeping all time-critical audio work isolated from display updates.

c. Hardware and Software Communication Approach

In Lab 6, hardware/software interaction relied on USB keycode handling and GPIO-style AXI peripherals. In Lab 7, display state is exposed through AXI4-Lite registers and memory map. This project benefits from the Lab 7 style for display because the MicroBlaze can update VRAM directly and is memory-mapped. The tradeoff is that BRAM ports and timing constraints become more critical.

2. Written Description of System

At a high level, the design runs three subsystems at the same time. Dedicated SD hardware initializes the SD card and streams raw audio sectors directly into BRAM. When ready, audio playback is then handled by a MicroBlaze interrupt service routine driven by the AXI timer, which reads the next sample from BRAM and writes it to the PWM audio peripheral at a fixed rate. The MicroBlaze main loop handles other tasks by processing USB keyboard input and updating the HDMI text controller display based on the most recent audio samples and the currently selected visualization mode.



System Block Diagram

b. Hardware SD Card Streaming to BRAM

Audio data is stored on the SD card as a raw byte stream with no filesystem. The sdcard_audio_streamer module initializes the card using SPI commands, then continuously reads 512-byte sectors and writes them byte-by-byte into BRAM. Unlike a one-time loader, this module streams indefinitely—the 8KB BRAM acts as a circular buffer that the SD streamer constantly refills while the CPU reads from another part of it. This allows a 3.47MB (157-second) audio file to play through only 8KB of memory.

A critical design constraint is rate matching. The SD card can read at roughly 80 KB/s, but the CPU only consumes audio at 22 KB/s (the sample rate). Without throttling, the SD streamer would overwrite data before the CPU could read it. We added a delay state (DELAY_ST) to the streaming state machine that waits approximately 22ms between 512-byte block reads, reducing throughput to about 23 KB/s—just slightly faster than consumption.

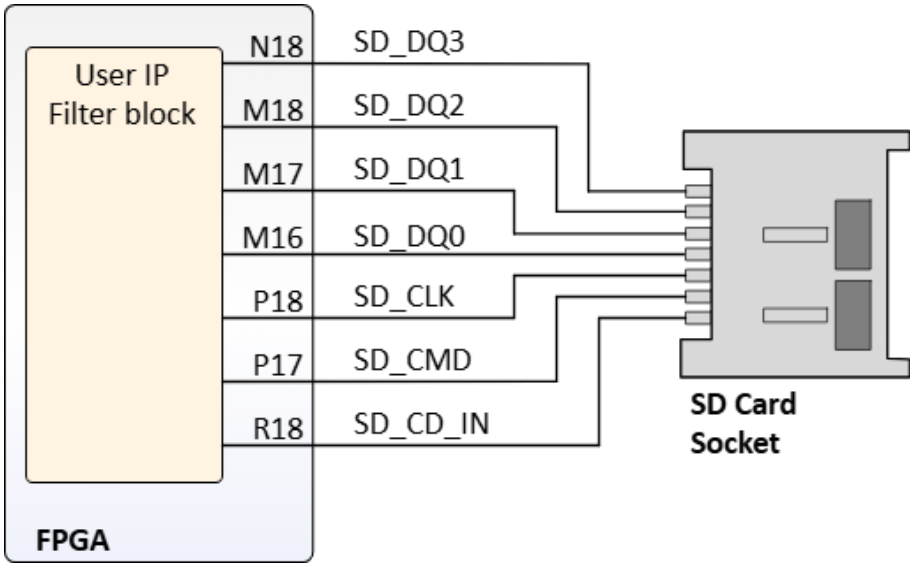
The SD logic requires a 50 MHz clock generated from the 100 MHz board clock. The BRAM is configured as true dual-port so the SD streamer can write through Port B while the CPU reads through Port A via the AXI BRAM controller.

c. Circular Buffer Operation

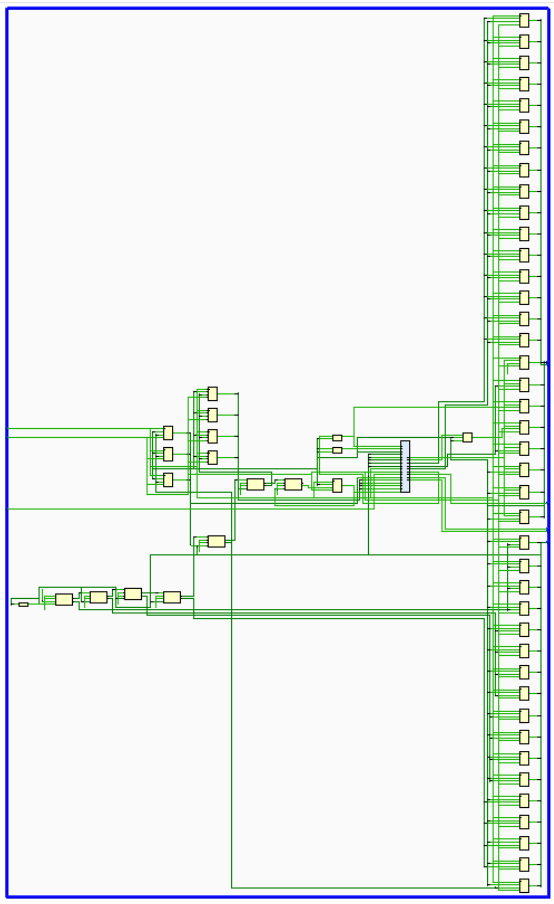
The 8KB BRAM operates as a circular buffer shared between the SD streamer (writer) and CPU (reader). The SD streamer maintains a write pointer that advances with each byte

written; when it reaches 8191, it wraps to 0 and continues. The CPU maintains a separate read pointer that also wraps at the buffer boundary.

For correct operation, the write pointer must stay ahead of the read pointer (so there's always data to play) but not so far ahead that it catches up and overwrites unplayed data. With the SD throttled to ~23 KB/s and CPU consuming at ~22 KB/s, the write pointer slowly gains on the read pointer but never overtakes it within the buffer size. This margin allows the system to handle small timing variations without audio glitches.



SD Card Interface on Urbana Board



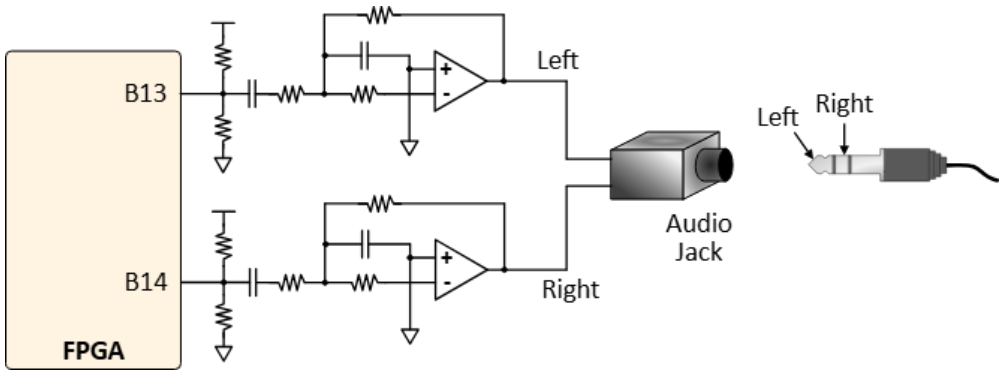
SD Reader Schematic

d. Audio Playback Pipeline (Timer ISR + PWM Output)

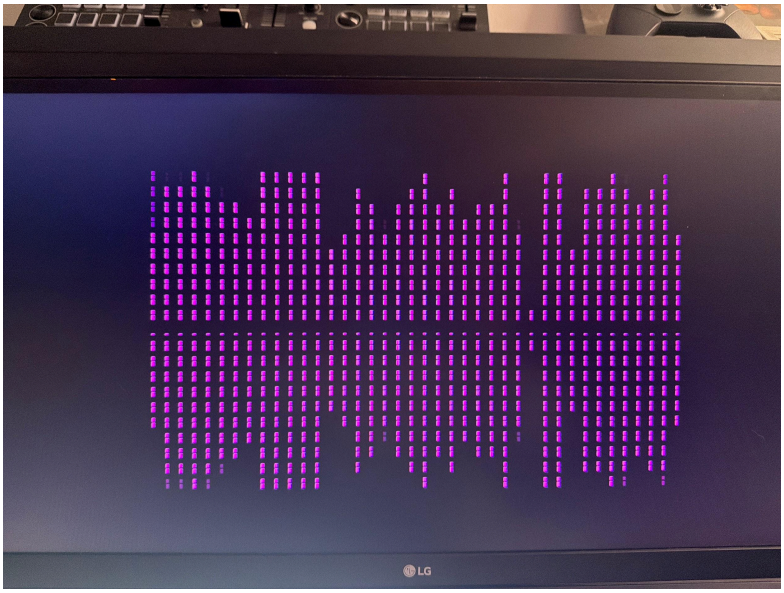
Audio playback is interrupt-driven because software polling can't achieve the precise timing required. For 22,050 Hz audio, samples must be output every 45.35 µs. Our early attempts used a polling loop with usleep() delays, but this caused audible distortion—usleep() has ±5-20 µs variation, and keyboard polling via SPI takes 50-500 µs, which completely breaks audio timing.

The solution was moving all audio output into a hardware timer ISR. The AXI timer generates interrupts at exactly 22,050 Hz. The ISR reads one byte from BRAM, applies volume scaling, writes to the PWM registers, advances the buffer position (with wraparound at 8192 bytes), and pushes the sample into a ring buffer for the visualizer. Total ISR execution time is about 1-2 µs.

The key insight is that interrupts preempt the main loop. Even if keyboard polling takes 5ms, the audio ISR still fires at exactly 22,050 Hz because the hardware timer doesn't care what the main loop is doing. This completely decouples audio timing from visualization and input handling.



Audio Port Interface on Urbana Board



Simple Format for Audio Wave

e. HDMI Visualization and Mode Selection

The display uses the Lab 7 HDMI text controller. The MicroBlaze writes characters and color attributes to VRAM over AXI. Five visualization modes are available: AUDIO (amplitude bars from actual audio), PULSE, WAVE, RANDOM, and SYMMETRIC (animated patterns). Modes are switched via USB keyboard (keys 1-5), with spacebar for pause and arrow keys for volume.

The main loop runs asynchronously from audio playback. Instead of using usleep() for timing (which would interfere with interrupts), the loop checks audio_tick_count—a counter incremented by every ISR call. When 367 ticks have passed (22050 Hz / 60 FPS ≈ 367), the loop updates the display. This gives approximately 60 FPS visualization without any blocking delays.

To avoid reading the sample ring buffer while the ISR is writing to it, the visualizer snapshots the buffer into a local array before processing. This prevents visual glitches from partially-updated data.

HDMI Output During Playback

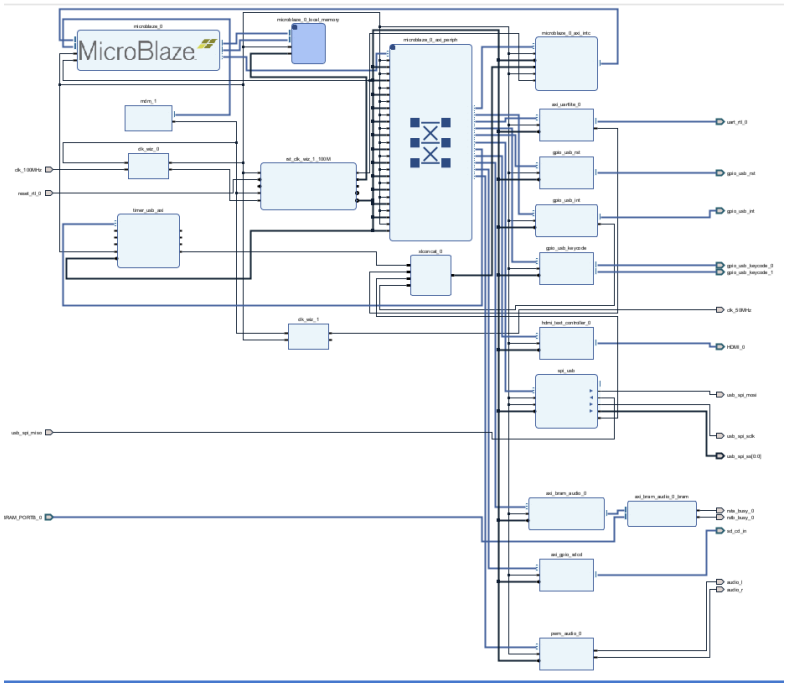
f. File Specifications

Parameter	**Value**
Audio File	Audio.raw, 3,470,849 bytes
Audio Format	8-bit unsigned PCM, mono
Sample Rate	22,050 Hz
Audio Duration	157 seconds
BRAM Buffer Size	8,192 bytes
Size Ratio	424:1 (file:buffer
SD Raw Throughput	~80 KB/s
SD Throttled Throughput	~23 KB/s
CPU Consumption Rate	22.05 KB/s
Time Interrupt Period	45.35 µs
Block Delay	1,100,000 cycles (22ms)

3. Block Diagrams

a. Top-Level System Block Diagram

The top-level module instantiates the MicroBlaze block design wrapper, the SD loader hardware, the audio BRAM, and the HDMI output path. The SD loader writes audio data into BRAM using one BRAM port, while the MicroBlaze reads the same BRAM via an AXI BRAM controller on the other port during playback.



System Block Diagram

b. Vivado Schematics

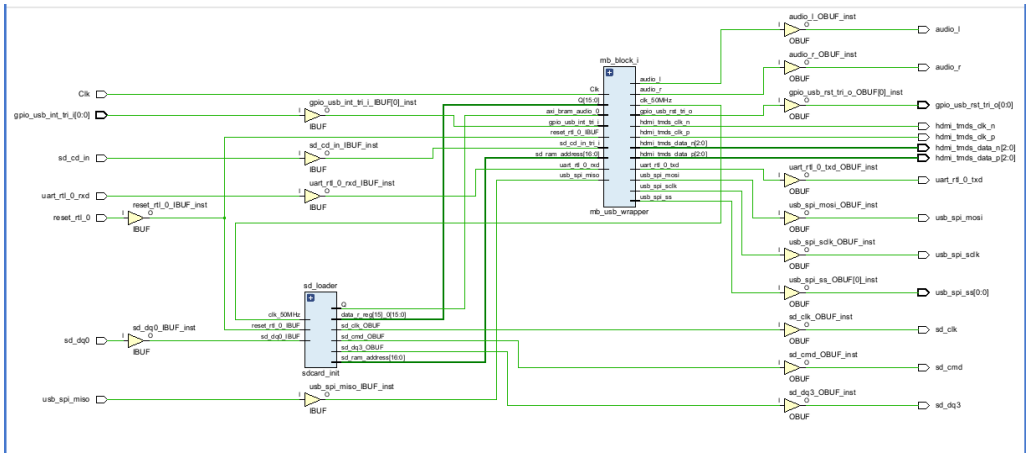
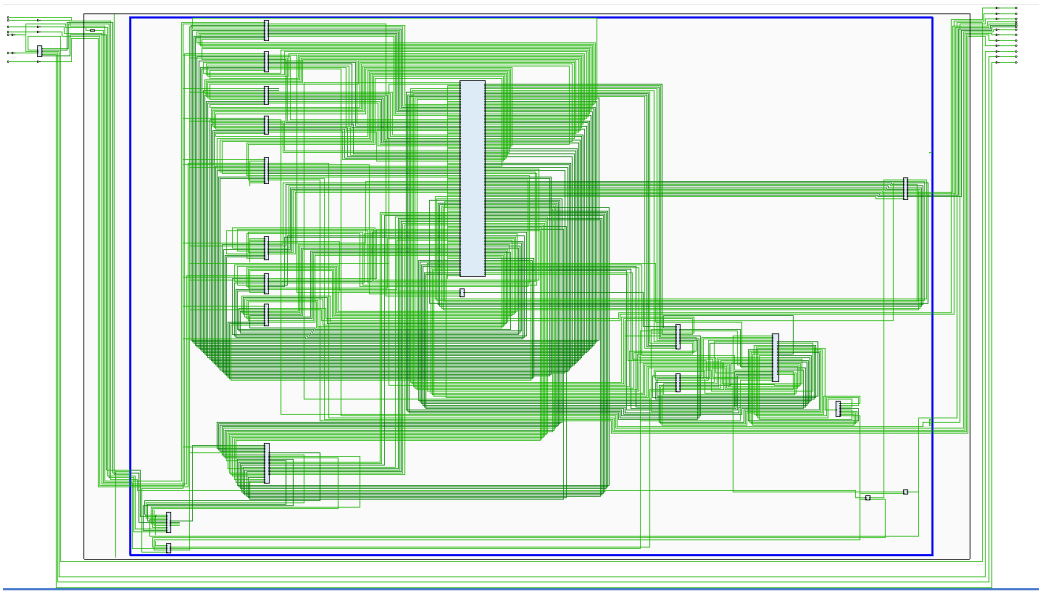


Figure 5: Overall Schematic



MicroBlaze Block Design Schematic

4. Module Descriptions

Module: mb_usb_hdmi_top.sv
Inputs: Clk (100 MHz), reset_rtl_0, uart_rtl_0_rxd, SD pins, USB host pins
Outputs: uart_rtl_0_txd, HDMI TMDS outputs, audio PWM outputs, USB/SD control pins
Description: Top-level wrapper that ties together the MicroBlaze block design, HDMI output pipeline, SD loader, and BRAM connections. Routes clocks/resets and ensures the SD loader has a valid write path into BRAM while the processor can read audio samples over AXI.
Purpose: Integrates all hardware and software-facing peripherals into a single system for audio playback and visualization.

Module: mb_block_i (mb_usb_wrapper)
Inputs: clocks/resets, UART, USB SPI signals, BRAM interface signals (as exposed by the block design)
Outputs: HDMI signals (through HDMI IP path), interrupt lines, and peripheral connections back to the top level
Description: Vivado-generated wrapper containing the MicroBlaze, AXI interconnect, interrupt controller wiring, timer, AXI BRAM controller, and memory-mapped peripherals used by software (USB host interface, HDMI text controller, audio output peripheral).
Purpose: Serves as the processor subsystem and AXI communication hub.

Module: sdcard_audio_streamer
Inputs: clk50, reset, enable, miso_i (SD data input)

Outputs: bram_we, bram_addr[31:0], bram_din[31:0], bram_we_bytes[3:0], cs_bo, sclk_o, mosi_o, ready, error, write_ptr[31:0], buffer_wrapped

Parameters:BRAM_SIZE_BYTES = 8192 (circular buffer size), SONG_LENGTH_BYTES = 3470849 (file size for loop detection), SDHC = 1'b0 (0 for standard SD ≤2GB, 1 for SDHC 4GB+), BLOCK_DELAY_CYCLES = 1100000 (throttle delay, 22ms at 50MHz)
State Machine: RESET_ST → READBLOCK → WAIT_BYTE → ACK_BYTE → WRITE_BYTE → (loop until block done) → DELAY_ST → READBLOCK
Description: Continuous streaming module that reads bytes from SD card and writes them to BRAM as a circular buffer. Unlike the original sdcard_init which loaded once and stopped, this module streams indefinitely. Writes one byte at a time using byte enables to avoid byte-ordering issues. Includes configurable throttling delay to prevent buffer overrun.
Purpose: Enables playback of audio files larger than BRAM by continuously streaming from SD card.

Module: Clock Wizard (100 MHz → 50 MHz)
Inputs: 100 MHz board clock, reset
Outputs: 50 MHz SD clock
Description: Generates the 50 MHz clock required by the SD loader logic so SD SPI timing matches the expected design assumptions.
Purpose: Ensures SD initialization and reads are reliable.

Module: audio_bram (blk_mem_gen)
Inputs: Dual-port BRAM signals (one port driven by SD loader writes, one port accessed via AXI BRAM controller for CPU reads)
Outputs: BRAM read data on each port
Description: Dual-port BRAM used as the shared audio sample buffer. One port supports sequential writes during SD loading, while the other supports CPU reads during playback.
Purpose: Stores audio samples with concurrent hardware write and software read access.

Module: AXI BRAM Controller (axi_bram_ctrl)
Inputs: AXI interface from MicroBlaze, BRAM port interface
Outputs: BRAM port transactions for CPU access
Description: Bridges the AXI bus to BRAM so that software can treat BRAM contents as memory-mapped data.
Purpose: Allows MicroBlaze code to read audio samples from BRAM using standard memory access.

Module: AXI Timer (axi_timer_0)
Inputs: AXI configuration interface, clock/reset

Outputs: periodic interrupt output
Description: Configured to generate periodic interrupts at the desired sample rate.
Purpose: Provides stable timing for interrupt-driven audio playback.

Module: AXI Interrupt Controller (axi_intc_0)
Inputs: interrupt lines from peripherals (timer, USB if used)
Outputs: single IRQ line to MicroBlaze
Description: Collects and prioritizes interrupts and presents them to the MicroBlaze interrupt input.
Purpose: Handles interrupt routing and software-visible interrupt status.

Module: PWM Audio Controller (custom AXI peripheral)
Inputs: AXI write interface from MicroBlaze
Outputs: audio_l, audio_r PWM signals
Description: Converts 8-bit sample values written by software into PWM output duty cycles on the audio outputs.
Purpose: Produces speaker/headphone-driveable PWM audio from digital samples.

5. Software Component Description

The software runs on the MicroBlaze and is split into two parts: a minimal ISR for sample-accurate playback, and a main loop for non-real-time work (USB polling, visualization updates, UI/status text).

a. Startup / Initialization Flow

The initialization order matters for stable audio. The system first initializes the platform and disables caches (required for correct BRAM access). It sets the PWM output to silence (value 128), then waits 500ms for the SD streamer to fill the BRAM buffer with initial data. The software checks if BRAM contains valid audio by counting non-zero/non-0xFF bytes—if fewer than 10% of the first 1000 bytes look like audio, playback is disabled.

Next, the USB keyboard is enumerated (this uses usleep() calls, which is fine before audio starts). The HDMI display is initialized with color palettes and the UI is drawn. Finally, the AXI timer and interrupt controller are configured and started. After XTmrCtr_Start() is called, the code never uses usleep() again—all timing comes from the interrupt counter. This prevents any blocking calls from interfering with the 22,050 Hz audio interrupts.

b. Timer ISR (Audio Playback)

On each timer interrupt, the ISR reads the next audio sample from BRAM and immediately writes it to the PWM audio left and right registers (mono replicated to both channels). It then advances the playback pointer and wraps it at the configured buffer length so playback continues

smoothly. Finally, the ISR pushes the sample into a small ring buffer used by the visualization routine. The ISR is intentionally kept minimal so the interrupt period stays consistent and audio timing does not drift.

c. Main Loop (Visualization + Keyboard Control)
The main loop continuously polls the USB keyboard and updates the system state (mode changes, pause/play, and any other controls implemented). Instead of using sleep calls, it relies on an audio-driven tick/counter to pace display refresh. When it is time to render a new frame, the program snapshots the ring buffer into a local array by briefly disabling interrupts, ensuring the frame is computed from a consistent set of samples. It then computes the visualization for the selected mode and writes the necessary VRAM/palette updates to the HDMI text controller over AXI.

d. SD Card Preparation (Software/Workflow Assumption)
Audio is converted to raw 8-bit PCM at 22,050 Hz and written to the SD card as a raw device image (no filesystem). Since the SD loader reads fixed sectors directly, the card must not be reformatted after imaging.

6. Testing and Debugging

We encountered four major bugs during development, each requiring a different debugging approach.

Bug 1: Audio looped every 3 seconds. The song would play for about 3 seconds, then jump back to the beginning regardless of song length. We wrote a diagnostic program that captured BRAM snapshots every 500ms and measured how often byte 0 changed. Expected: ~27 changes in 10 seconds (8KB buffer ÷ 22KB/s). Actual: 97 changes. The SD streamer was writing 3.6x faster than the CPU was reading, overwriting data before it could be played.
Fix: Added a DELAY_ST state to the streaming state machine with a 1,100,000-cycle delay (22ms at 50MHz) between 512-byte block reads. This throttled SD throughput from ~80 KB/s to ~23 KB/s.

Bug 2: Wrong SD card addressing mode. Our 2GB microSD card is a standard SD card, but the code had SDHC = 1'b1. Standard SD expects byte addresses (0, 512, 1024...) while SDHC expects block numbers (0, 1, 2...). This caused garbage reads after the first few blocks.
Fix: Changed to SDHC = 1'b0.

Bug 3: Corrupted byte ordering. After fixing the streaming rate, BRAM data showed lots of 0x00 and 0xFF values instead of valid audio (which should be centered around 0x80). The

original code packed two bytes into 16-bit words before writing to 32-bit BRAM, causing byte-swap issues.

Fix: Rewrote the streamer to write one byte at a time using byte enables:

```
assign bram_din = {byte_data, byte_data, byte_data, byte_data};
case (bram_byte_addr[1:0])
   2'b00: bram_we_bytes = 4'b0001;
   2'b01: bram_we_bytes = 4'b0010;
   2'b10: bram_we_bytes = 4'b0100;
   2'b11: bram_we_bytes = 4'b1000;
endcase
```

Bug 4: "Crunchy" audio during keyboard/display updates. Even with correct streaming, audio had audible glitches when pressing keys or during visualization updates. The original polling loop used usleep(45) for timing, but usleep() varies by ±5-20 µs. Keyboard polling via SPI added 50-500 µs of unpredictable delay. For 22,050 Hz audio, we need 45.35 µs precision—any variation causes distortion.

Fix: Moved audio output into a hardware timer ISR. The AXI timer fires at exactly 22,050 Hz regardless of what the main loop is doing. The ISR takes only 1-2 µs. The main loop handles the keyboard and display without affecting audio timing.

7. Design Resources and Statistics

LUT	3979
DSP	3
Memory (BRAM)	36
Flip-Flops	3595
Frequency	104.16 MHz
Static Power	0.075 W
Dynamic Power	0.496 W
Total Power	0.571 W

This design uses a significant amount of BRAM due to storing max sized audio samples on-chip and enabling simultaneous SD writes + CPU reads. DSP use is minimal because visualization is not FFT-based and playback uses a PWM peripheral rather than DSP processing.

8. Conclusion

This project implements continuous SD-to-BRAM audio streaming with interrupt-driven playback and real-time visualization on the Urbana Spartan-7 platform. The main challenge was playing a 3.47MB file through 8KB of BRAM, which required the SD streamer and CPU to run simultaneously with matched rates.

The trickiest bug to find was the streaming rate mismatch; without the diagnostic tool that measured BRAM update frequency, we wouldn't have realized the SD was writing 3.6x faster than the CPU was reading. The most important lesson was that real-time audio requires hardware timing; software delays like usleep() simply aren't precise enough for 22,050 Hz sample output.